

Building a Software Requirements Specification and Design for an Avionics System: An Experience Report

Andrés Paz

École de Technologie Supérieure, Université du Québec
Montreal, Canada
andres.paz-loboguerrero.1@ens.etsmtl.ca

Ghizlane El Boussaidi

École de Technologie Supérieure, Université du Québec
Montreal, Canada
ghizlane.elboussaidi@etsmtl.ca

ABSTRACT

As with many of the products and systems that pervade us, aircraft rely more and more on software for controlling the behaviour of their systems. In consequence, the field has seen increased work around more up-to-date, effective software engineering technologies for aiding avionics software providers in reducing software and development complexities and supporting them in their certification endeavours. However, there is a lack in the literature of reusable, comprehensive references about avionics software developments in conformance with DO-178C. Moreover, there is a need for a benchmark specification to support the evaluation of proposed engineering approaches in the field. This paper presents a software development case study of an avionics control software for a landing gear system. All the documentation for the software's requirements specification and design has been developed to conform with the DO-178C guideline and the applicable DO-331 and DO-332 supplements for model-based and object-oriented development, respectively. A requirements specification and design methodology is proposed and followed for the construction of the software in the case study. Furthermore, the paper discusses the observations, and challenges and issues experienced throughout the process.

CCS CONCEPTS

•Software and its engineering →Requirements analysis; Software design engineering;

KEYWORDS

Requirements specification, software design, avionics software, DO-178C, DO-331, DO-332, case study, experience report, landing gear control software

ACM Reference format:

Andrés Paz and Ghizlane El Boussaidi. 2018. Building a Software Requirements Specification and Design for an Avionics System: An Experience Report. In *Proceedings of SAC 2018: Symposium on Applied Computing, Pau, France, April 9–13, 2018 (SAC 2018)*, 10 pages.
DOI: 10.1145/3167132.3167268

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

SAC 2018, Pau, France

© 2018 ACM. 978-1-4503-5191-1/18/04...\$15.00

DOI: 10.1145/3167132.3167268

1 INTRODUCTION

Software is becoming pervasive in aircraft, controlling more and more the behaviour of critical avionics systems. Avionics software development is largely driven towards compliance with the DO-178C guideline [20] (and its European equivalent ED-12C). Development of DO-178C-compliant avionics software begins when a set of high-level system requirements and system safety requirements is allocated to software. Such requirements must be developed (*i.e.* refined and decomposed) into high-level software requirements (HLRs) and documented in the Software Requirements Data *data item*. A review of the HLRs validates these against the system and safety requirements allocated to software (SRATS) to check if their intent is being accurately captured by the HLRs in a way suitable for directing the software design activities. The software design process covers the definition of the software architecture and the specification of low-level software requirements (LLRs). LLRs together with the software architecture are developed from the specified HLRs and are used to guide the implementation of the source code. The Design Description *data item* communicates both the software architecture and the LLRs. Supplemental documents to DO-178C, DO-331 [10] and DO-332 [12], modify these guidelines to support model-based and object-oriented developments, respectively.

There has been increasing work around more up-to-date, effective software engineering technologies to aid avionics software providers in reducing software and development complexities and to support them in their DO-178C certification endeavours [14]. However, the legal and safety implications that public scrutiny may bring onto avionics industry manufacturers make them keen on keeping their projects confidential. This situation hinders research and education on the engineering and certification of avionics software. It is, therefore, necessary to have detailed and open documentation of software for airborne systems that is readily available for research or education.

A number of studies have supported their work with examples and case studies of avionics systems that involve some piece of software (*e.g.*, [8, 21–23]). Others report on the development of avionics systems that rely on software for controlling the behaviour of such systems (*e.g.*, [3, 18]). Apart from most of them not being openly available, the case studies, as described in the previous works, do not allow for their reuse as comprehensive references for avionics software development in conformance with DO-178C. The case study by Boniol *et al.* in [3] of a landing gear system is significantly detailed and constitutes a good starting point for reducing the lack of reference avionics software documentation intended for research and education purposes.

This paper, thus, builds on the descriptions given in the case study by Boniol *et al.* to present an avionics software case study for a landing gear control software (LGCS) developed in compliance with the DO-178C guideline, and the DO-331 and DO-332 supplements. The descriptions presented here for the LGCS have addressed inconsistencies, ambiguities and confusing wording found in Boniol *et al.*'s case study. The LGCS case study also follows the set of recommended practices on requirements engineering and management from [7] and the work from Blouin in [2]. The goal of this paper is threefold. First, developing a detailed requirements specification and design of an avionics software that is representative in both complexity and constraints found in the industry. Second, making this specification open and available for further research in the field. And third, building a requirements specification that serves as a benchmark avionics software specification for supporting the evaluation of different (existing and new) model-based approaches. A methodology following DO-178C, DO-331 and DO-332 has been proposed to develop the requirements specification and a design that meets such requirements. Observations, and challenges and issues experienced throughout the process are reported in this paper.

The remainder of this paper is organized as follows. Section 2 provides an overview of the software considerations in airborne systems and equipment certification covered by DO-178C, and summarizes related work. Section 3 presents the methodology followed to develop the software requirements specification and design for the case study. Section 4 describes the specification and design of the software for the avionics system case study. Section 5 discusses lessons learned, challenges and issues of building such specification and design for a software of an avionics system. Section 6 discusses threats to validity. Finally, Section 7 concludes the paper and outlines future work.

2 BACKGROUND AND RELATED WORK

2.1 DO-178C Software considerations in airborne systems and equipment certification

The aim of DO-178C is to produce software that is validated and verified for its airworthiness, *i.e.* reliability and safe-to-use in flight. DO-178C is a conceptual guideline identifying the set of *best practices* to take into consideration during the development of software for airborne systems and equipment. These *best practices* are stated in the form of *objectives*, which have to be achieved by carrying out a set of explicitly defined *activities* that will output the acceptable evidence (*e.g.*, plans, requirements, design description), known as *data items*.

DO-178C prescribes a software life cycle comprised by the following three process groups: (i) software planning process, (ii) software development processes that include software requirements, software design, software coding and software integration, and (iii) transverse processes that include software verification and validation, software configuration management, quality assurance and certification liaison. The focus of this paper lies in a subset of the software development processes group, namely the software requirements and software design processes producing the HLRs, and software architecture and LLRs, respectively.

DO-178C maintains the core text of its predecessor, DO-178B, with some needed corrections to errors and inconsistencies, and improvements to interpretability, but most importantly, it addresses new considerations and practices in key areas of contemporary software development. The new considerations and practices have been produced and published as separate, supplement documents referring to, or modifying content of the main DO-178C document where applicable, as well as including additional terminology and content where necessary. Two of the supplement documents are of particular interest in the context of the construction of the proposed case study: DO-331 and DO-332.

DO-331 [10] contains development and verification guidelines when parts of the software are to be expressed using *models*. Selic [19] defines a model as a reduced (*i.e.* simplified, abstract) representation of some (aspect of a) system that highlights its properties of interest from a given viewpoint. With DO-331, models may be used for software analysis, verification, simulation or code generation. Two types of models are addressed: *specification models* and *design models*. The former represent HLRs and the latter represent LLRs and/or software architecture. Even though DO-331 supports the use of models, it does not alleviate the objectives defined in the main document, therefore, as models represent requirements and/or software architecture they must be treated as such. Models may coexist alongside textual requirements specifications.

DO-332 [12] contains guidelines when the software, or parts of it, are to be coded using object-oriented technologies (OOTs). Although this supplement is particularly intended for the software coding and integration processes, compliance with its guidance also becomes relevant when using object-oriented modelling languages like UML, which provide constructs for representing OOT features, *e.g.*, inheritance, polymorphism, overloading. DO-332 has various modifications to the main guidelines in DO-178C due to the shift in the programming paradigm from procedural languages. OOTs are founded on the principle of strong abstraction where the programming unit, an *object* (defined by a *class*), holds both the data and the *methods* (or procedures) to manipulate such data. Class hierarchies and the interactions among the instantiated objects are what delivers the expected functionality of the system. DO-332 includes specific development compliance needs covering class hierarchies. In particular, DO-332 suggests class hierarchies should be derived from HLRs, and bidirectional traces should be defined between HLRs and methods, since it is in the latter where the former are implemented.

2.2 Related work

Existing studies related to this work can be divided into (i) generic work in the field of safety-critical software requirements engineering and design that support their findings and evaluations with an avionics system case study or example, and (ii) existing reports on the development of avionics systems that rely on software for controlling the behaviour of such systems.

A number of studies have supported their work with examples and case studies of avionics systems that involve some piece of software (*e.g.*, [8, 21–23]). Leveson *et al.* [8] describe a requirements specification language for process-control systems. They apply their proposed language on an industrial aircraft Traffic Collision

Avoidance System level II (TCAS II). The resulting requirements model was adopted by the FAA as the official requirements specification for TCAS II. The specification, thus, had to be developed with industrial quality and in accordance with the applicable regulation at the time (*i.e.* DO-178B). However, the TCAS II specification is not openly available for research. Furthermore, the authors do not expand on the aspects of compliance with regulation.

Zoughbi *et al.* [23] and Wu *et al.* [22] propose UML profiles with the objectives of capturing safety-related requirements allocated to software and enabling the monitoring of the design and implementation of the software with regard to the provided safety requirements. The profiles also serve to define a set of design guidelines for avionics software. Both studies illustrate their proposals through industrial avionics systems. Neither of these case studies is openly available for research. Additionally, the descriptions given for the case studies are missing important details about the inputs and outputs of the systems. Also, the provided software design only corresponds to the software architecture and lacks the behavioural specification, *i.e.* the LLRs. Furthermore, in the case of Zoughbi *et al.*'s work, their interest is to discuss on the software requirements specification and design in the context of DO-178B, which is now an outdated revision of DO-178 and superseded by DO-178C.

White *et al.*'s work in [21] is closely related to the work presented in this paper. It falls into the first category of related work, although, it has some elements of the second category. On one hand, they propose a set of criteria for assisting in the construction of a requirements specification in compliance with DO-178C. The authors validate their approach on an example requirements specification for the display of the fuel quantity in an aircraft. Their HLRs and LLRs specifications, however, have some shortcomings as some of the LLRs presented are, in fact, HLRs. Presenting them as LLRs makes the specification of the HLRs go against the characteristics of good requirements specification since they are incomplete and, thus, the presented LLRs could not have been developed from them. On the other hand, the authors report on some of the difficulties that arise when developing a requirements specification for an avionics software under DO-178C. The difficulties they describe are focused on the capturing of derived requirements, a special type of requirement under DO-178C that is characterized by not being directly traceable to higher-level requirements because it gives behaviour beyond what is specified in the higher-level requirements.

Only a small number of reports on the development of avionics systems that rely on software have been found in the literature (*e.g.*, [3, 18]). Schamai *et al.* [18] develop a requirements specification for the design of the secondary flight system of an aircraft. Particularly, the authors describe the spoiler activation requirements. However, the case study is not openly available for research either. Moreover, although the system heavily relies on software for its operation, as it is intended for a fly-by-wire aircraft, the requirements that are presented are not developed taking into account the DO-178C guideline.

Boniol *et al.* [3], on their part, have introduced a case study of a landing gear system. Their descriptions provide a detailed view of a conventional tricycle landing gear system, and the detailed structure and inner workings of the elements involved, specially the hardware elements. However, despite being both complex and representative of industrial needs, the software requirements given

are not developed following the DO-178C guideline and, thus, fail to present the detailed reference expected in software development for airborne systems. Boniol *et al.*'s descriptions have been taken to build the detailed reference for the landing gear control software case study presented in this paper.

3 REQUIREMENTS SPECIFICATION AND DESIGN METHODOLOGY FOR THE CASE STUDY

To the best of the authors' knowledge, no methodology for building software requirements specification and design following DO-178 guidelines has been reported in the literature. Therefore, as part of the construction of the case study, a methodology for such purpose was defined. This methodology encompasses the general flow for software requirements specification and design defined in DO-178C. It consists of three major activities, as illustrated in Figure 1: *Develop HLRs*, *Develop Software Architecture* and *Develop LLRs*. The activities have a sequential nature as promoted by DO-178C to allow for a meticulous engineering that builds a safe product. Nevertheless, the activities have their actions organized to form iterative and incremental cycles that will gradually build their outputs until they yield a software requirements specification and design that is suitable for directing the following phases in the development process, namely coding and verification. Furthermore, as a means for quality assurance, several practitioners from the avionics industry with ample experience in avionics software development and DO-178C certification participated in validating the outputs of the activities throughout the iterations.

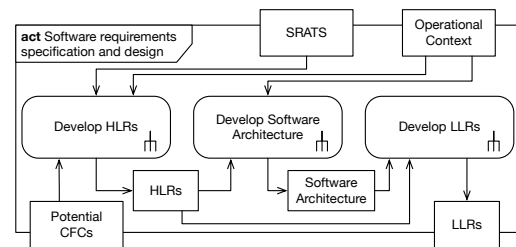


Figure 1: General flow for requirements specification and design.

Avionics systems are routinely specified in natural language [2, 15, 18]. The methodology, thus, considers system requirements allocated to software (SRATS) and high-level requirements (HLRs) to be captured using natural language. The practices of the FAA's¹ Requirements Engineering Management Handbook [7] and the work from Blouin in [2] are followed to restrict the grammar and vocabulary used for the textual specification. The handbook suggests that in order to reduce ambiguities, improve readability, and facilitate the analysis and review of the requirements, these should be specified as statements of how the software will change a set of *controllable variables* (*i.e.* values in the operational context the software can directly affect) in response to changes in a set of *monitorable variables* (*i.e.* values in the operational context the software responds to).

¹U.S. Federal Aviation Administration

Once a set of system requirements is allocated to software, an operational context for the software is defined and a set of potential software contributions to failure conditions (CFCs, identified during the software planning) has been determined, the software requirements specification, *i.e.* the *Develop HLRs* activity, begins. Figure 2 expands the *Develop HLRs* activity. It is necessary to first perform a manual review of the SRATS in order to verify them for ambiguities, inconsistencies or undefined conditions. If the SRATS need any clarification or correction, it is requested to the system processes and the activity waits for the clarified or corrected SRATS. SRATS may be specified with enough detail to carry on with the software design; in which case they are redefined as the HLRs. If the SRATS lack refinement or they do not preclude the CFCs, they are iteratively and incrementally developed into HLRs that accurately capture the intent of the SRATS and preclude the CFCs. Traces between SRATS and HLRs must be established. However, if a trace cannot be established directly, then the HLR is labeled as a derived requirement.

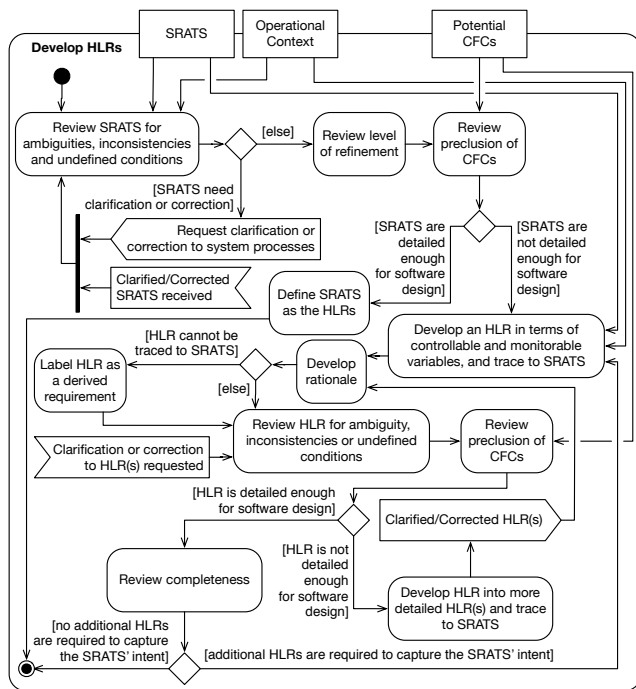


Figure 2: Expanded view of the *Develop HLRs* activity.

The software design in the proposed methodology must produce design models as defined in DO-331. Hence, to maximize adherence to the guidance provided by both DO-178C, DO-331 and DO-332, the software design has been segregated as recommended by [17] into two different activities, *Develop Software Architecture* and *Develop LLRs*, that work at two distinct levels. The first level, the *Develop Software Architecture* activity represents the architecture as software components and their relationships. Software design under DO-178C guidelines must enable a black-box verification of the system's behaviour. For this to happen, DO-178C highlights the importance of avoiding introduction of complexity during the

design process. Hence, the *Develop Software Architecture* activity seeks to incorporate principles of software engineering that help minimize complexity and promote verifiability, such as modularity and encapsulation [1]. A component-based design approach using UML components and realizing classes was defined for the *Develop Software Architecture* activity as a result.

Figure 3 expands the *Develop Software Architecture* activity. The activity starts by identifying or defining an architectural style in accordance with the expected functionality of the software to be built and its operational context. Then, the software components that will realize the functionality expressed in the HLRs, as well as the dependencies between them, are identified and defined based on such architectural style. Each component is allocated to a subset of the HLRs. Software design patterns may be identified too as work progresses. When all responsibilities have been assigned to components and no additional components are required to cover the HLRs, the activity continues with the identification of the class hierarchies realizing each of the components. Every class must be allocated to its originating HLRs, which are a subset of the ones associated to its containing component. Clarifications or corrections to HLRs may be requested to the *Develop HLRs* activity and the *Develop Software Architecture* activity waits for the clarified or corrected HLRs to be returned.

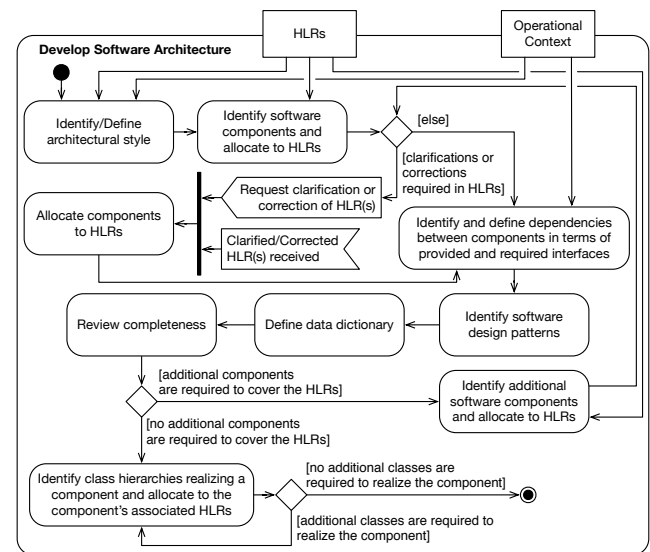


Figure 3: Expanded view of the *Develop Software Architecture* activity.

The second level, the *Develop LLRs* activity, focuses on representing low-level requirements and the functionalities that the realizing classes of every software component will implement. In line with the objective of developing a benchmark avionics software specification that can be used for evaluating different model-based approaches, the *Develop LLRs* activity has two alternate definitions: one that outputs textual LLRs and another one that outputs LLRs as a DO-331 design model. Figure 4 expands the former and Figure 5 expands the latter. This also serves to illustrate how the

methodology can be tweaked to adapt it to different development scenarios.

Having established the architecture initiates the work in the *Develop LLRs* activity. For every realizing class defined in the architecture LLRs are specified in terms of the class's controllable and monitorable variables. Each LLR is allocated to its originating HLRs, which are a subset of the ones associated to the class. If the allocation is not possible due to required clarifications or corrections in the HLRs, these are requested to the *Develop HLRs* activity and the *Develop LLRs* activity waits for the clarified or corrected HLRs to be returned in order to continue. If the allocation is not possible because the LLR cannot be traced to HLRs, then the LLR is labeled as a derived LLR. The activity ends when source code can be directly implemented for all realizing classes without any further refinement, and all of them have their behaviour specified.

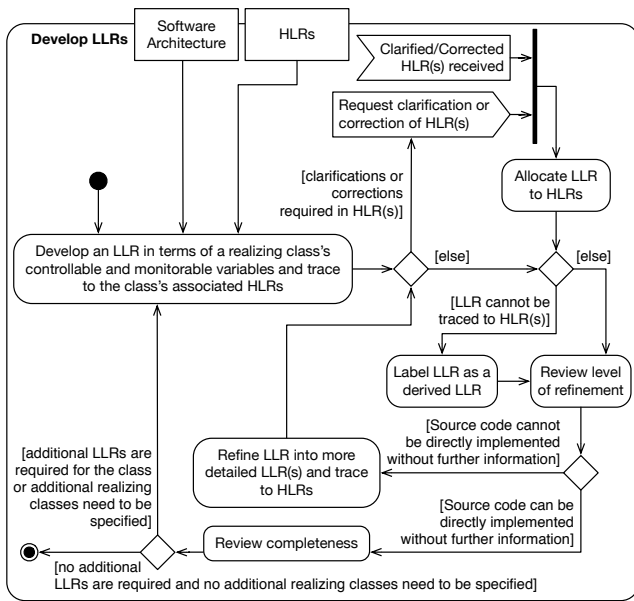


Figure 4: Expanded view of the *Develop LLRs* activity for specifying textual LLRs.

The alternate definition for the *Develop LLRs* activity that outputs a design model (see Figure 5) has a similar workflow, however, for every realizing class defined in the architecture a state machine is defined describing its behaviour instead. The state machine's elements are allocated to their originating HLRs as well.

UML state machines were chosen since they are an important part of UML for modelling the behaviour of objects. DO-178C and the proposed methodology also prompted a set of essential features for the modelling of LLRs, which UML state machine models are able to cover: (i) layered modelling and hidden decompositions, (ii) factorization of commonalities or reuse of modelled elements, (iii) partial ordering and concurrent flow of control, (iv) algorithms, (v) time observation and timing constraints, (vi) interruptions in the flow of control and exception handling, (vii) explicit interactions between distinct system parts, (viii) complex trigger conditions and triggered actions, and (ix) flow of data (usage, production and

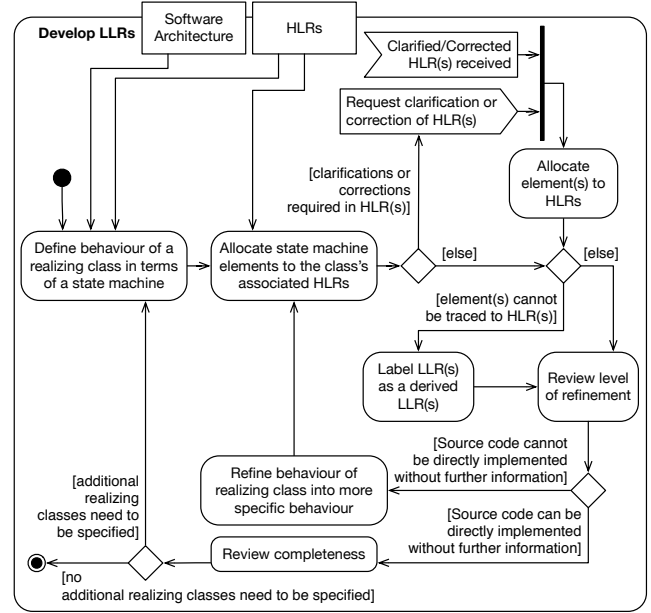


Figure 5: Expanded view of the *Develop LLRs* activity for specifying LLRs as state machine models.

storage). Furthermore, UML state machines were also chosen because of their close proximity to MathWorks's Stateflow, which is commonly used for modelling real-time/mission critical systems. Such relationship may facilitate their integration as part of a future extension to the LGCS case study to add more modelling mechanisms as it is a requirement for effective system modelling [6, 16] and commonplace in avionics providers.

4 CASE STUDY DESCRIPTION: THE LANDING GEAR CONTROL SOFTWARE

The complete case study of the Landing Gear Control Software (LGCS) has been made available online in the report found in [13]. The document was iteratively built by the authors, who are experienced in software specification and development, and verified by several domain experts. The document is organized in chapters, each of which corresponds to one of the DO-178C data items output by the planning, software requirements and software design processes. This section briefly presents some excerpts from the chapters that were created following the methodology in Section 3. Subsection 4.1 summarizes the LGCS operating context, while Subsections 4.2, 4.3 and 4.4 highlight notable elements for discussion resulting from the execution of the three activities in the methodology described in Section 3.

4.1 Case study: Operational context

The overall system is a landing gear system whose undercarriage is retractable and arranged in a conventional tricycle configuration. The undercarriage is composed of two main wheel assemblies situated one under each wing of the aircraft and a nose wheel assembly at the front of the aircraft. The wheel assemblies, or gears, are

retracted into compartments inside the wings and fuselage of the aircraft and concealed behind doors after the aircraft has taken off and began the climb phase. Conversely, at the beginning of the landing phase the gears must be extended. The doors covering the wheel compartments need to be open prior to and closed after any retraction or extension is performed.

The behaviour of the landing gear system is managed by the LGCS that runs in a digital controller. Figure 6 illustrates the operational context of the LGCS. The aircraft's cockpit accommodates the system's pilot interface. From it, the pilots provide the desired gear position to the system through a gear lever. A color-coded light set indicator provides them with information about the state of the gears and the system itself. All the gears and doors are moved through the sequential actuation (*i.e.* opening and closing) of five hydraulic electro-valves (EV) that control the oil pressure in the hydraulic circuit to which the gears and doors are connected. One general electro-valve pressurizes the hydraulic circuit and the remaining four specific electro-valves (*i.e.* a door opening, a door closing, a gear retraction and a gear extension electro-valves) move their attached mechanical part. Seventeen sensors are in charge of monitoring the state of the various mechanical parts (*i.e.* one analogical switch sensor, one hydraulic circuit pressure sensor, three door open sensors, three door closed sensors, three gear retracted sensors, three gear extended sensors and three gear shock absorber sensors). For redundancy purposes, all sensors perform three independent readings describing simultaneously the same situation. At least two of the three readings must have an equal value for the LGCS to trust the sensor.

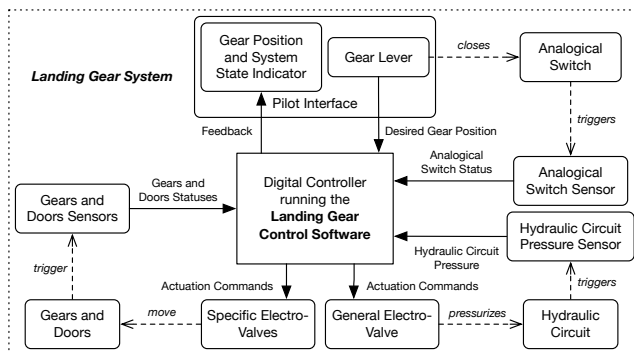


Figure 6: LGCS operational context.

The LGCS focuses on two primary needs: (i) to sequence the commands that actuate the hydraulic electro-valves for the movement of the gears and doors to the given desired gear position taking into account the inputs from the different sensors, and (ii) to display visible cues indicating the state of the gears and the system through the indicator in the pilot interface. The following are some examples of the potential software contributions to failure conditions (CFCs) that were determined for the LGCS during planning.

- CFC-2 The LGCS generates actuation commands when a sensor is providing invalid data.
- CFC-3 The LGCS generates actuation commands when an electro-valve is not functioning.

4.2 Case study: High-level requirements

Being the LGCS based on the case study presented in [3], the descriptions found in such work relating to the digital controller have been taken as the SRATS. However, since the SRATS contained ambiguities, inconsistencies and undefined conditions they had to be corrected and improved before developing the HLRs. It took three iterations of SRATS and HLRs development to obtain a favorable perception about them from the industrial practitioners involved. Table 1 shows a subset of the HLRs developed from the SRATS. This subset of requirements will be used to show some of the LLRs that were developed from them.

4.3 Case study: Software architecture

Figure 7 presents the resulting software architecture as a UML component diagram. The names of the interfaces are omitted to keep the figure uncluttered. The LGCS receives input from different sensors and acts upon those inputs to drive the actuation of the hydraulic electro-valves that will move the gears into the desired position. This context of operation matches the *Process Control* architectural style [9], where a system uses a set of inputs to determine a set of outputs that will produce a new state of the environment. Thus, the LGCS's architecture follows the *Process Control* architectural style.

Interfacing with the various sensors is the *SensorManager* component. The *SensorManager* component receives the inputs from the sensors using pull communication through the interface with the corresponding *Sensor* component. The *SensorManager* component performs a validation of the data received from each sensor before passing it along on to the *SequenceController* component. The *SensorManager* component will report a failure of a sensor to the *OperatingModeManager* component when an invalid sensor data is received.

The *SequenceController* component sequences the actuation commands according with the inputs received from the *SensorManager* and the *PilotInterfaceManager* components. The *PilotInterfaceManager* component interfaces with the pilot interface to output the feedback. The *EVManager* component interfaces with the hydraulic electro-valves to output the actuation commands. The *OperatingModeManager* component will place the LGCS into a failed state if any failure is detected, and the system will remain in such a state thereafter. The allocation of HLRs to components in Figure 7 is done by means of UML comments. Each of the identified components is realized by one class.

4.4 Case study: Low-level requirements

Two parallel specifications of LLRs were created, namely a textual specification using natural language and a model-based specification using UML state machines. It also took three iterations of each form of the LLRs specifications to obtain a favorable perception from the industrial practitioners involved. Table 2 shows a subset of the textual LLRs developed from the presented HLRs.

Figure 8 shows a fragment of an equivalent model-based specification as a UML state machine for LLR-35, LLR-43, LLR-44, LLR-45, LLR-56 and LLR-57. The allocation of HLRs to elements of the UML state machine fragments in the figure is done by means of UML comments.

Table 1: Examples of HLRs for the LGCS

ID	Description	Rationale	Traces	Precluded CFCs
HLR-1	When the LGCS receives data from one of the LGS sensors, the LGCS shall process the three Readings associated to the sensor data based on the following rules...(the rules are not shown due to length constraints).	A sensor should not be trusted if it cannot accurately describe the status of its associated system entity. For redundancy purposes, all the sensors perform three independent readings describing simultaneously the same situation. Thus, voting strategy is followed to determine the overall value and validity of the sensor. At least two of the three readings must be equal.	SRATS-1	CFC-2
HLR-6	Once the overall value of the Hydraulic Circuit Pressure is greater than or equal to 30,000 kPa and less than 35,000 kPa after the General EV Actuation Command is set to Open, the LGCS can set to Open the necessary specific EV.	This is the specified oil pressure needed in the hydraulic circuit for operating the specific electro-valves.	SRATS-6 LLR-14 LLR-43 LLR-44 LLR-45	
HLR-7	Once at least 0.2 seconds have elapsed since the General EV Actuation Command was set to Open, the LGCS can set to Open the Door Opening EV Actuation Command.	This is the specified minimum wait time between stimulations of the general electro-valve and the specific electro-valves due to oil pressure differential produced by the change in fluid velocity across the hydraulic circuit.	SRATS-7 LLR-14 LLR-46	
HLR-4	When the LGCS is currently executing a retraction sequence and a Down value is received for the Desired Gear Position, the LGCS shall halt the current retraction sequence and revert all the actions that were executed. Likewise, when the LGCS is currently executing an extension sequence and an Up value is received for the Desired Gear Position, the LGCS shall halt the current extension sequence and revert all the actions that were executed.	A gear motion can be canceled by the pilot or copilot at any step of an ongoing sequence. Any action executed of the ongoing sequence has to be reverted.	SRATS-4 LLR-35	
HLR-12	Once 2 seconds have elapsed since the General EV Actuation Command was set to Open and the overall value of the Hydraulic Circuit Pressure is still less than 30,000 kPa, the LGCS shall detect a failure of the general hydraulic electro-valve and halt the currently executing sequence.	The hydraulic circuit should be pressurized in less than 2 seconds after the general electro-valve has been stimulated. The hydraulic circuit being unpressurized after this time is an indication of its failure. If a failure is detected the system should not be not be trusted to perform correctly and, therefore, the LGCS shall halt its operation.	SRATS-12 LLR-44 LLR-56 LLR-57	CFC-3

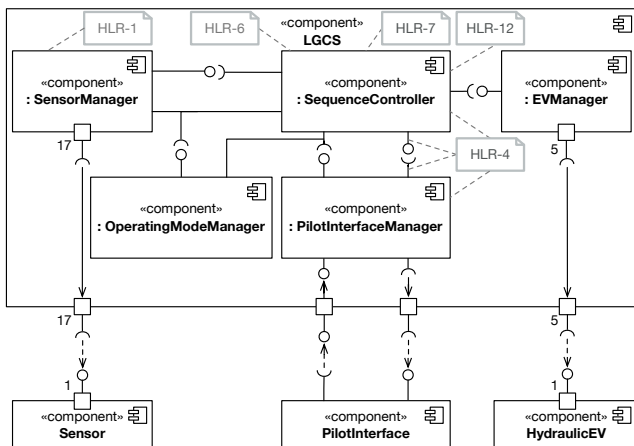


Figure 7: Architecture for the LGCS.

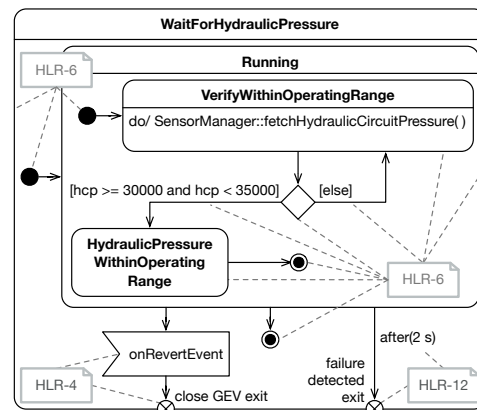


Figure 8: UML state machine fragment for the SequenceController component.

Table 2: Examples of LLRs for the LGCS

ID	Description	Traces	Component
LLR-14	If the <code>PressurizationEvent</code> and the <code>DelayDOEVActuationTimeoutEvent</code> are raised, the <code>Door Opening EV Actuation Command</code> shall be set to <code>Open</code> and the <code>waitForDoorsOpen</code> method shall be activated.	HLR-2 HLR-3 HLR-6 HLR-7	SequenceController
LLR-43	If the <code>General EV Actuation Command</code> is set to <code>Open</code> , the <code>waitForHydraulicPressure</code> method shall be activated.	HLR-6	SequenceController.
LLR-44	If the <code>waitForHydraulicPressure</code> method is active and the overall value of the <code>Hydraulic Circuit Pressure</code> monitorable variable is less than 30,000 kPa, the <code>waitForHydraulicPressure</code> method shall remain active until the <code>PressurizationTimeoutEvent</code> is raised.	HLR-6 HLR-12	SequenceController
LLR-45	If the <code>waitForHydraulicPressure</code> method is active and the overall value of the <code>Hydraulic Circuit Pressure</code> is greater than or equal to 30,000 kPa and less than 35,000 kPa, the <code>waitForHydraulicPressure</code> method shall end and the <code>PressurizationEvent</code> shall be raised.	HLR-6	SequenceController
LLR-46	If the <code>General EV Actuation Command</code> was set to <code>Open</code> , the <code>DelayDOEVActuation</code> method shall be activated.	HLR-7	SequenceController
LLR-56	If the <code>waitForHydraulicPressure</code> method is active and 2 seconds have elapsed since the <code>General EV Actuation Command</code> was set to <code>Open</code> , the <code>PressurizationTimeoutEvent</code> shall be raised.	HLR-12	SequenceController
LLR-57	If the <code>waitForHydraulicPressure</code> method is active, the <code>PressurizationTimeoutEvent</code> is raised and the overall value of the <code>Hydraulic Circuit Pressure</code> monitorable variables is less than 30,000 kPa, the <code>FailureEvent</code> shall be raised.	HLR-12	SequenceController
LLR-35	If the <code>waitForHydraulicPressure</code> method is active and the <code>RevertEvent</code> is raised, all the actions that were previously executed shall be reverted.	HLR-4	SequenceController

5 LESSONS LEARNED, CHALLENGES AND ISSUES

This section discusses and elaborates on some lessons learned, and challenges and issues experienced for each of the activities performed to build the LGCS case study. This section also highlights some limitations of the case study and outlines steps to overcome them and extend the case study.

5.1 Quality and granularity of SRATS

As mentioned in Section 4.2, the SRATS were taken from the descriptions found in [3]. The descriptions, however, were scattered throughout the text, which hindered the task of developing the HLRs and establishing traceability links between the SRATS and HLRs. Furthermore, the SRATS' review in the *Develop HLRs* activity, found them to contain several inconsistencies, ambiguities and confusing wording. Hence, the SRATS had to be corrected, clarified and uniquely identified first before proceeding with the development of the HLRs. The heavy reliance on review actions in the *Develop HLRs* activity is intended to guarantee the requirements are clear. It is imperative that both sets of SRATS and HLRs are reviewed to be at an acceptable quality level.

While working on the SRATS, it was observed that these can be, in fact, very detailed so as to guide the software's design without any further refinement into HLRs. Feedback on the matter from the practitioners involved points out that such a situation is not infrequent in the industry because the SRATS are intended to give the software development team everything they need to know to develop the software. The more clear, precise and complete the

SRATS are, the easier the job of the software development team becomes. In the LGCS case study, the SRATS were not developed to a refinement level that could lead directly to software design, as it was sought to have a separation between the SRATS and the HLRs in order to make the case study as rich as possible.

5.2 Requirements specification language

SRATS and HLRs are routinely specified in natural language, textual documents using, e.g., Microsoft Office Word documents or Microsoft Office Excel spreadsheets, and managed with textual requirements databases like IBM Rational DOORS [2, 15]. The SRATS, HLRs and LLRs of the LGCS were specified in natural language, as in the examples of Section 4.2. However, natural language is not a suitable form of specification for supporting requirements-based analyses and verification due to its inherent ambiguities [11]. Thus, a form of controlled natural language was used for their specification. As suggested by [7], SRATS, HLRs and LLRs were written using the names of monitorable and controllable variables, and system and software architectural elements. Having the requirements formulated in this way brought important advantages in terms of the properties of a good requirements specification (e.g., consistency, completeness). Subsequent stages of the development of the LGCS may also see benefits from this form of specification such as the generation of test cases. Future work on the case study also plans to incorporate the use of DO-331 specification models to address these concerns for HLRs from a model-based perspective in a similar way as it was done with the LLRs.

Representing the LLRs in UML state machine models posed its challenges as they stressed the authors' knowledge and understanding of the UML notation and its semantics, and the notation and semantics themselves. The UML specification is plagued with omissions, inconsistencies, vaguenesses and comprehensibility problems in the UML metamodel and in the definition of its semantics [4]. Hence, the UML specification should not be taken on its own as a standard for model development under DO-178C, DO-331 and DO-332. It is necessary that the modelling standard defines any additional information, constraints and specificities that help remove any notational and semantical unclarity of UML.

A long overdue improvement of UML state machines surfaced during the *Develop LLRs* activity. UML state machines capture trigger conditions and triggered actions as annotations on the transition arrows. The notation broke down when developing some of the LLRs (e.g., LLRs developed from HLR-1). The combination of diagrammatic and tabular notations has been suggested to facilitate the specification and analysis of such complex trigger conditions [5], but requires extending UML with non-standardized constructs.

5.3 Granularity of LLRs

In the *Develop LLRs* activity, it was observed that the operating context of the LGCS originated a fair amount of intertwined conditions that the software had to respect at any given moment. Therefore, developing the LLRs for the LGCS with an appropriate level of granularity was challenging. DO-178C expects LLRs to be very detailed in order to enable source code to be implemented without the need for more information. This may be misleading towards wanting to express conditions as close as possible to the code, say in pseudocode or in an action language (e.g., UML Alf). Unlike the situation with SRATS and HLRs where these could be considered the same, DO-178C requires a greater separation between LLRs and code than pseudocode / action languages are able to provide. The reason lies with the need to enable black-box verification of the software's behaviour. Since the differences between pseudocode / action languages and actual implementation code may not be discernible in all cases, black-box verification may not be possible to achieve.

5.4 Bi-directional traces in model-based LLRs

Both the *Develop Software Architecture* and the *Develop LLRs* activities are required to allocate HLRs to the elements of their resulting design models. In the case study this was achieved through the use of UML comments. Sarkis *et al.* [17] recommend maintaining such traceability links in the design models themselves and that the use of comment blocks is sufficient for the task. This certainly works for supporting backwards traceability, *i.e.* linking LLRs to their source HLRs, when the latter are uniquely identifiable. However, the same cannot be said for forwards traceability, *i.e.* linking HLRs to their developed model-based LLRs, which is also required by DO-178C. In order to enable requirements-based verification of the software, and give proper visibility to requirements when verifying coverage and assessing the impact of change, all requirements need to be uniquely identifiable. Identifying what can be an LLR in UML state machines is not trivial and not possible without an extension to UML. One suggestion is to consider an LLR as a 4-tuple made up

of an origin state, a destination state, a transition from the origin state to the destination state, and a set of actions triggered and completed during such transition. The definition becomes more complex if fork, join and choice pseudostates are taken into account. The same issue may appear also in specification models capturing non-text-based HLRs.

6 THREATS TO VALIDITY

Certain issues may threaten the validity of the resulting software requirements specification and design for the LGCS. These issues are related to the quality of the resulting requirements and the language used for their specification. This section briefly presents these issues and the way each of them was mitigated. (1) The authors are experienced in software specification and design but do not have broad experience in avionics. Hence the reliance on in-depth discussions with industrial practitioners that have expertise in both avionics development and DO-178C certification. (2) System requirements and operational context were defined on the basis of the case study from Boniol *et al.* Thus, the quality of these artifacts is dependent on the descriptions obtained from such case study. Nonetheless, significant efforts were made to address any inconsistencies, ambiguities and confusing wording that were found. (3) The granularity of requirements was defined by the authors. However, a discussion with industrial practitioners took place after each iteration that led to refinements. (4) System requirements and HLRs have been specified using natural language, which may have resulted in ambiguities and inconsistencies within the text. In an attempt to mitigate such situation, a form of controlled natural language was introduced for their specification. Further efforts will be made in future work with the use of specification models.

7 CONCLUSION AND FUTURE WORK

This paper presented an avionics software case study of a landing gear control software (LGCS) and the observations, challenges and issues experienced during its construction following the DO-178C guideline, and its DO-331 and DO-332 supplements. The observations, challenges and issues described are limited to a subset of the software development processes, namely the software requirements and software design processes. The complete case study of the Landing Gear Control Software (LGCS) has been made available in the report found in [13].

Avionics software case studies have been mostly kept confidential due to legal and safety implications that public scrutiny of commercial systems may bring onto avionics industry manufacturers. This situation hinders research and education on the engineering and certification of avionics software. Therefore, there is a need of detailed and open documentation of software for airborne systems. The LGCS case study is a work in progress, but accessible for the analysis of different engineering problems across the industry and the evaluation of proposed solutions.

The observations, and encountered challenges and issues can be summarized in four points: (i) *Quality and granularity of SRATS*. As a major observation with regards to the development of HLRs is that the level of quality and refinement of the SRATS affects the amount of effort for the activity. It should be sought to specify the SRATS as clearly and as detailed as possible since it will certainly remove

considerable effort and, thus, reducing development time and costs. (ii) *Requirements specification language*. The use of specification models developed in compliance with DO-331 may significantly improve the comprehensibility and analyzability of HLRs over the industry's *de facto* standard of employing natural language. Developing the model-based LLRs for the LGCS was difficult. The intertwined conditions in the HLRs stressed the UML specification's power of expression and suitability as a modelling standard under DO-331. In one case calling for an extension. (iii) *Granularity of LLRs*. There must be a clear separation between LLRs and code in order to enable black-box verification of the implementation. (iv) *Bi-directional traceability in model-based LLRs*. Regarding traceability between requirement decompositions, the use of UML comments was sufficient for backwards traceability when HLRs are text-based. Establishing bi-directional traces in UML models appeared as an issue due to the impossibility of uniquely identifying LLRs in UML. Bi-directional traceability is an important component for DO-178C certification as it is key for supporting verification and analysis activities, and remains an open challenge.

Some opportunities for improvement of the case study arise from the discussion held in this paper. First, the extension of the methodology to include an activity for software verification and validation that will derive verification cases from the specified HLRs and execute them against the design to verify normal-range and robustness conditions, as well as coverage of all software requirements. Second, the development of HLRs and their representation in specification models, which will provide a scenario for the usage and analysis of this type of models. Third, the need to incorporate a heterogeneity of modelling mechanisms as it is required for effective system modelling. And fourth, enrich the case study with other requirements, such as those regarding how the software will be deployed.

ACKNOWLEDGMENTS

The research leading to this paper has been supported by the Consortium for Research and Innovation in Aerospace in Quebec (CRIAQ) under project AVIO-604. The authors also thank those AVIO-604 partners who provided input for and feedback on the development of the LGCS case study.

REFERENCES

- [1] M. Bialy, V. Pantelic, J. Jaskolka, A. Schaap, L. Patcas, M. Lawford, and A. Wassyng. Software engineering for model-based development by domain experts. In *Handbook of System Safety and Security*. Syngress, 1st edition, 2017.
- [2] D. Blouin. *Modeling languages for requirements engineering and quantitative analysis of embedded systems*. PhD thesis, Université de Bretagne-Sud, Dec. 2013.
- [3] F. Boniol and V. Wiels. *The landing gear system case study*. In *ABZ 2014: The Landing Gear Case Study*. Springer, 2014.
- [4] H. Fecher, J. Schönborn, M. Kyas, and W.-P. d. Roever. *29 new unclarities in the semantics of UML 2.0 state machines*. In *Formal Methods and Software Engineering*. Springer, 2005.
- [5] M. Herrmannsdörfer and B. Berenbach. Tabular notations for state machine-based specifications. *The Journal of Defense Software Engineering*, 2008.
- [6] E. A. Lee. *Disciplined heterogeneous modeling*. In *Model Driven Engineering Languages and Systems: 13th Int. Conf. MODELS 2010, Oslo, Norway, October 3-8, 2010, Part II*. Springer, 2010.
- [7] D. L. Lempia and S. P. Miller. *Requirements Engineering Management Handbook*. Technical report, Federal Aviation Administration (FAA), Washington, DC, USA, 2009.
- [8] N. G. Leveson, M. P. E. Heimdahl, H. Hildreth, and J. D. Reese. Requirements specification for process-control systems. *IEEE Trans. Softw. Eng.*, 20(9):684–707, Sept. 1994.
- [9] W. S. Levine. *The control handbook*. The electrical engineering handbook series. CRC Press New York, 1996.
- [10] Model-Based Development and Verification Supplement to DO-178C and DO-278A. Standard DO-331, Radio Technical Commission for Aeronautics (RTCA), Inc., 2011.
- [11] Y. Moy, E. Ledinet, H. Delseny, V. Wiels, and B. Monate. Testing or Formal Verification: DO-178C Alternatives and Industrial Experience. *IEEE Software*, 30(3):50–57, May 2013.
- [12] Object-Oriented Technologies and Related Techniques Supplement to DO-178C and DO-278A. Standard DO-332, Radio Technical Commission for Aeronautics (RTCA), Inc., 2011.
- [13] A. Paz and G. El Boussaidi. Landing Gear Control Software: An avionics software development case study. Technical report, École de Technologie Supérieure, Montreal, Canada, 2017. URL: <http://dx.doi.org/10.13140/RG.2.2.34900.19848>.
- [14] R. G. Pettit, N. Mezcciani, and J. Fant. On the needs and challenges of model-based engineering for spaceflight software systems. In *2014 IEEE 17th Int. Symp. on Object / Component / Service-Oriented Real-Time Distributed Computing*, 2014.
- [15] B. Potter. Complying with DO-178C and DO-331 using Model-Based Design. Technical report, MathWorks, 2012.
- [16] N. Rozanski and E. Woods. *Software Systems Architecture: Working With Stakeholders Using Viewpoints and Perspectives*. Addison-Wesley Professional, second edition. Edition, 2011.
- [17] A. Sarkis and L. A. V. Dias. A set of rules for production of design models compliant with standards DO-178C and DO-331. In *Int. Conf. on Inf. Tech.: New Generations*, 2014.
- [18] W. Schamai, L. Buffoni, N. Albarello, P. Fontes De Miranda, and P. Fritzson. An aeronautic case study for requirement formalization and automated model composition in modelica. In *Proceedings of the 11th Int. Modelica Conf., Versailles, France, September 21-23, 2015*, number 118, pages 911–920, 2015.
- [19] B. Selic. The pragmatics of model-driven development. *IEEE Software*, 20(5):19–25, Sept. 2003.
- [20] Software Considerations in Airborne Systems and Equipment Certification. Standard DO-178C, Radio Technical Commission for Aeronautics (RTCA), Inc., 2011.
- [21] J. White and H. Reza. Deriving DO-178C requirements within the appropriate level of hierarchy. In *The 7th Int. Conf. on Software Engineering Advances*, Lisbon, Portugal, Nov. 2012.
- [22] J. Wu, T. Yue, S. Ali, and H. Zhang. A modeling methodology to facilitate safety-oriented architecture design of industrial avionics software. *Software: Practice and Experience*, 45(7):893–924, 2015.
- [23] G. Zoughbi, L. Briand, and Y. Labiche. Modeling Safety and Airworthiness (RTCA DO-178B) Information: Conceptual Model and UML Profile. *Softw. Syst. Model.*, 10(3), 2011.